## Table of Contents


Faculty of Engineering ... 3
Department of Computer and Systems Engineering ... 3
1. Abstract ... 4
2. Introduction ... 5
3. Dataset Description and Problem Framing ... 6
3.1 Dataset Source and Structure ... 6
3.2 Classification Type and Expected Challenges ... 6
4. Data Cleaning and Preprocessing ... 7
4.1 Initial Loading and Inspection ... 7
4.2 Fixing Undocumented Category Codes ... 7
4.3 Outlier Handling via Winsorization ... 7
4.4 Train / Validation / Test Split ... 7
4.5 Feature Scaling ... 7
4.6 Class Imbalance Correction via SMOTE ... 8
5. Feature Engineering ... 9
6. Exploratory Data Analysis ... 11
6.1 Class Distribution ... 11
6.2 Payment Status Distribution and Default Rate ... 11
6.3 Bill Amount Outlier Analysis ... 12
6.4 Engineered Feature Correlations ... 13
6.5 Revolving User Default Rate ... 13
7. Model Development ... 14
7.1 Logistic Regression ... 14
7.2 Decision Tree ... 14
7.3 k-Nearest Neighbors (kNN) ... 15
7.4 Shallow Neural Network (MLP) ... 15
8. Performance Evaluation ... 16
8.1 Validation Set Results ... 16
8.2 Final Test Set Results ... 16
8.3 Confusion Matrices ... 16
8.4 ROC Curves ... 18
9. Model Comparison and Selection ... 19
9.1 Overall Comparison ... 19
9.2 Decision Tree — Best Overall ... 19
9.3 kNN — High Recall but Poor Precision ... 19

9.4 Logistic Regression — Competitive Baseline ..... 19
9.5 Neural Network — Good Potential, Needs Tuning ..... 19
9.6 Complexity and Deployment Considerations ..... 20
10. Error Analysis and Limitations ..... 21
10.1 Types of Errors ..... 21
10.2 Characterizing Missed Defaulters (False Negatives) ..... 21
10.3 Dataset Limitations ..... 21
10.4 Model-Specific Limitations ..... 22
10.5 Future Improvements ..... 22
11. AI Tool Usage Reflection ..... 23
11.1 Feature Engineering Brainstorming (Claude, Anthropic) ..... 23
11.2 SMOTE Data Leakage Warning (Claude, Anthropic) ..... 23
11.3 Scaler Selection (Claude, Anthropic) ..... 23
12. Team Contributions ..... 24
13. Conclusion ..... 25

# Ain Shams University

**Faculty of Engineering**

Department of Computer and Systems Engineering


# Credit Card Default Prediction

*A Supervised Machine Learning Classification Study*

**CSE374 — Machine Learning**

Milestone 2 — Final Report


**Team Members**

Mena Moheb Abdelshaheed · George Ibrahim Abdo · Abdalla Ragaee Ahmed


May 2026

*Dataset: Default of Credit Card Clients — UCI Machine Learning Repository*

# 1. Abstract

This report presents a complete supervised machine learning classification study aimed at predicting credit card payment defaults among Taiwanese bank clients. The dataset, sourced from the UCI Machine Learning Repository, contains 30,000 records with 23 original features capturing six months of payment history, bill statements, and demographic information. The target variable is binary: whether a client will default on their next payment.

The study involved extensive data cleaning, domain-driven feature engineering resulting in ten new behavioral features, outlier handling via winsorization, class imbalance correction using SMOTE, and feature scaling with RobustScaler. Four classifiers were implemented: Logistic Regression, Decision Tree, k-Nearest Neighbors (kNN), and a Shallow Neural Network (MLP). All models were evaluated on a held-out test set using accuracy, precision, recall, F1-score, and AUC-ROC.

The Decision Tree achieved the best overall performance with an AUC-ROC of 0.760 and an F1-score of 0.510 for the default class on the test set. A deliberate decision threshold of 0.40 (rather than the default 0.50) was used across all models to prioritize recall — minimizing missed defaulters — which is the operationally critical objective in a credit risk context.

## 2. Introduction

Credit risk assessment is one of the most consequential applications of machine learning in finance. When a bank extends credit, it accepts the risk that the borrower may fail to repay. The ability to predict which clients are likely to default — before it happens — allows financial institutions to intervene early: adjusting credit limits, offering payment restructuring plans, or flagging accounts for manual review. Failing to identify a true defaulter (a false negative) results in unrecovered debt and direct financial loss, while excessive false positives impose unnecessary friction on otherwise healthy clients.

This project frames the problem as a binary classification task. Using the Default of Credit Card Clients dataset from Taiwan, published on the UCI Machine Learning Repository, we aim to build and compare a range of classification models that predict whether a client will default in the following month based on their demographic profile and the previous six months of financial behavior.

The dataset presents several realistic challenges that are common in financial machine learning: significant class imbalance (approximately 22% of clients defaulted), skewed financial distributions, correlated features across monthly time steps, and noisy ordinal codes in the payment status variables. Addressing these challenges through principled preprocessing and feature engineering is a central focus of this study.

This report documents the complete pipeline from raw data through model comparison and error analysis, following the requirements of the CSE374 course project specification.

# 3. Dataset Description and Problem Framing

## 3.1 Dataset Source and Structure

The dataset was obtained from the UCI Machine Learning Repository under the title "Default of Credit Card Clients (Taiwan)." It was originally collected by Yeh and Lien (2009) to study predictors of default among credit card clients of a Taiwanese bank in 2005. The dataset contains 30,000 client records and 24 columns: 23 input features and one binary target variable.

| Group | Features | Description |
|---|---|---|
| Demographics | SEX, EDUCATION, MARRIAGE, AGE | Client profile information |
| Credit Limit | LIMIT_BAL | Approved credit limit in New Taiwan Dollars |
| Payment Status | PAY_0 to PAY_6 | Monthly repayment status: -2 to 8 (ordinal) |
| Bill Amounts | BILL_AMT1 – BILL_AMT6 | Monthly bill statement amount (NT$) |
| Payment Amounts | PAY_AMT1 – PAY_AMT6 | Actual payment made each month (NT$) |
| Target | default | 1 = defaulted next month, 0 = did not default |

*Table 3.1: Feature groups in the credit card default dataset*

## 3.2 Classification Type and Expected Challenges

This is a binary classification problem: the model must assign each client to one of two classes — default (1) or no default (0). The following challenges were anticipated and addressed during preprocessing and modeling:

- Class imbalance: Approximately 22.1% of clients defaulted, creating a 78:22 class ratio. A naive classifier predicting "no default" for all samples would achieve 78% accuracy while being entirely useless for the bank's actual objective.
- Right-skewed financial distributions: Bill and payment amounts are heavily right-skewed with extreme outliers — a small number of clients have balances or payments in the millions. These values can distort distance-based models (kNN) and gradient-based learners (MLP).
- Noisy payment codes: The PAY_X features use an ordinal encoding where values 0 and -2 are not documented in the original dataset description but appear frequently. These were investigated and handled during cleaning.
- Feature correlation: Bill amounts across six months are highly correlated (a client's balance changes gradually). This redundancy motivated feature engineering over raw feature selection.
- Borderline cases: Financially stressed clients who manage to avoid default are very similar in feature space to those who do default — creating an inherently difficult classification boundary.

# 4. Data Cleaning and Preprocessing

## 4.1 Initial Loading and Inspection

The dataset was loaded directly from the UCI repository using pandas. The Excel file uses row 0 as a label description and row 1 as the true header, requiring the header=1 parameter. The ID column, which serves only as a row identifier with no predictive value, was dropped immediately. No missing values were found in the dataset — all 30,000 records were complete, which is common for administrative financial records.

## 4.2 Fixing Undocumented Category Codes

The EDUCATION feature contains values 0, 5, and 6 that are not defined in the dataset's official documentation. These undocumented categories were merged into category 4 ("Other"), which is the dataset's designated catch-all. Similarly, the MARRIAGE feature contains value 0, which is also undocumented and was merged into category 3 ("Other"). This prevents the model from treating these codes as meaningful distinct categories when they are effectively noise labels.

## 4.3 Outlier Handling via Winsorization

Bill and payment amount columns were found to contain extreme outliers. For example, BILL_AMT1 had a maximum value of 964,511 NT$ with a standard deviation of 73,636 — indicating substantial right skew. Rather than removing these rows (which would reduce the dataset and potentially introduce sampling bias), winsorization was applied: values below the 1st percentile or above the 99th percentile were capped at those respective bounds.

After winsorization, BILL_AMT1's maximum dropped to 350,111 NT$ and the standard deviation decreased to 69,305, confirming effective outlier reduction. No rows were removed and the dataset shape remained (30,000, 24). The 1st/99th percentile choice was deliberate: it preserves legitimate financial variation while trimming only the most extreme values where data quality is uncertain.

## 4.4 Train / Validation / Test Split

The dataset was split into three non-overlapping partitions using stratified sampling to preserve the 22.1% default rate in every subset:

- Training set: 21,000 samples (70%) — used for model fitting
- Validation set: 3,000 samples (10%) — used for hyperparameter tuning and model selection
- Test set: 6,000 samples (20%) — held out until final evaluation; never used for any modeling decisions

Stratified splitting ensures that the class imbalance is consistently represented across all three sets, preventing any artificial differences in difficulty between training and evaluation environments. The test set was strictly held out and not examined until Section 7 of this report.

## 4.5 Feature Scaling

RobustScaler was applied to all features. Unlike StandardScaler (which uses mean and standard deviation), RobustScaler centers data using the median and scales using the interquartile range (IQR). This makes it significantly less sensitive to remaining outliers in the financial columns — even after winsorization, some asymmetry remains in the distributions.

The scaler was fitted exclusively on the training set and then used to transform the validation and test sets. This is a critical anti-leakage measure: fitting the scaler on all data would allow information from the test set to influence the scaling parameters, producing overly optimistic results.

## 4.6 Class Imbalance Correction via SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) was applied exclusively to the training set after splitting and scaling. SMOTE creates synthetic samples of the minority class by interpolating between existing minority class samples in feature space, producing a balanced training distribution of 16,355 samples per class (50/50 split).

Crucially, SMOTE was not applied to the validation or test sets, which retain the original 22.1% default rate. Applying SMOTE before splitting — or applying it to the validation/test sets — would constitute data leakage, artificially inflating performance metrics. Additionally, class_weight='balanced' was used in all applicable classifiers as a secondary layer of imbalance correction.

# 5. Feature Engineering

The original 23 features describe individual monthly snapshots in isolation. However, default risk is fundamentally about behavioral patterns over time. A client who was late once is very different from one who is consistently late; a client paying 90% of their bill is far healthier than one paying 10%. Ten new features were engineered to capture these behavioral patterns explicitly. Each feature carries a clear financial interpretation.

| Feature | Formula / Derivation | Financial Rationale |
|---|---|---|
| max_delay | max(PAY_0 ... PAY_6) | Worst single payment delay — isolates peak risk signal |
| num_delays | count(PAY_X > 0) | Frequency of late payments — severity vs. consistency |
| avg_pay_status | mean(PAY_0 ... PAY_6) | Typical repayment behavior over the 6-month window |
| total_bill | sum(BILL_AMT1–6) | Total credit exposure over 6 months |
| total_paid | sum(PAY_AMT1–6) | Total repayment made over 6 months |
| payment_ratio | total_paid / (\|total_bill\| + 1) | Fraction of billed amount actually repaid — ability to repay |
| util_rate | BILL_AMT1 / (LIMIT_BAL + 1) | Credit utilization — high ratio signals financial stress |
| avg_bill_change | mean(month-to-month bill differences) | Debt trend — rising bills indicate accumulating risk |
| paid_vs_billed | PAY_AMT1 / (\|BILL_AMT1\| + 1) | Recent payment coverage — most predictive recent signal |
| revolving | 1 if never fully paid any bill in 6 months | Structural revolving user — higher default rate observed (25.5% vs 15.5%) |

*Table 5.1: Engineered features with financial interpretation*

The total feature count increased from 23 to 33 after engineering. Correlation analysis confirmed that the new features carry meaningful signal: max_delay and num_delays showed the strongest positive correlation with the default target, while payment_ratio and paid_vs_billed were negatively correlated (clients who pay more of their bills default less often). The revolving flag revealed a practically significant split: revolving users defaulted at 25.5%, compared to 15.5% for clients who fully paid their balance at least once.

All derived features were verified through correlation analysis before inclusion. Features with very low or near-zero correlation were candidates for exclusion, but all ten engineered features showed statistically meaningful signal.

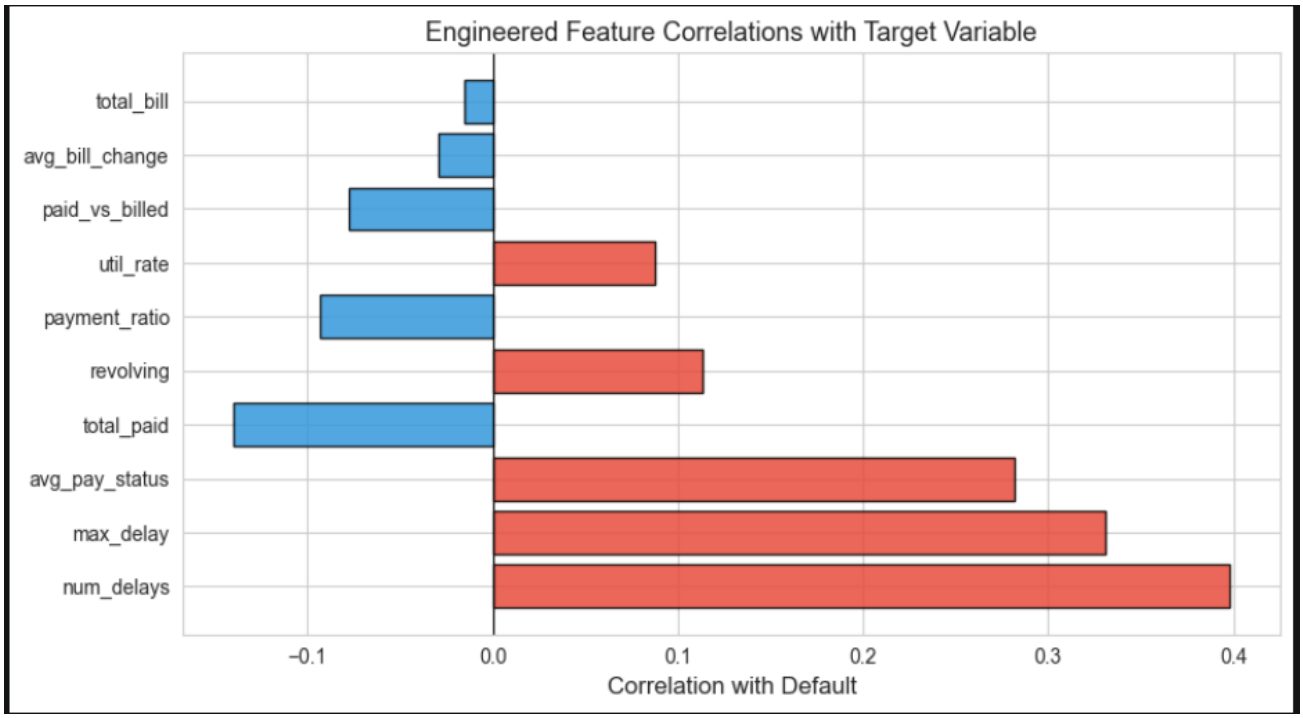


*Figure 5.1: Pearson correlation of engineered features with the default target variable*

# 6. Exploratory Data Analysis

Exploratory analysis was conducted before and after feature engineering to build understanding of the data, validate preprocessing decisions, and identify the most informative signals for modeling. Visualizations were selected to directly support modeling choices rather than serve as aesthetic additions.

## 6.1 Class Distribution

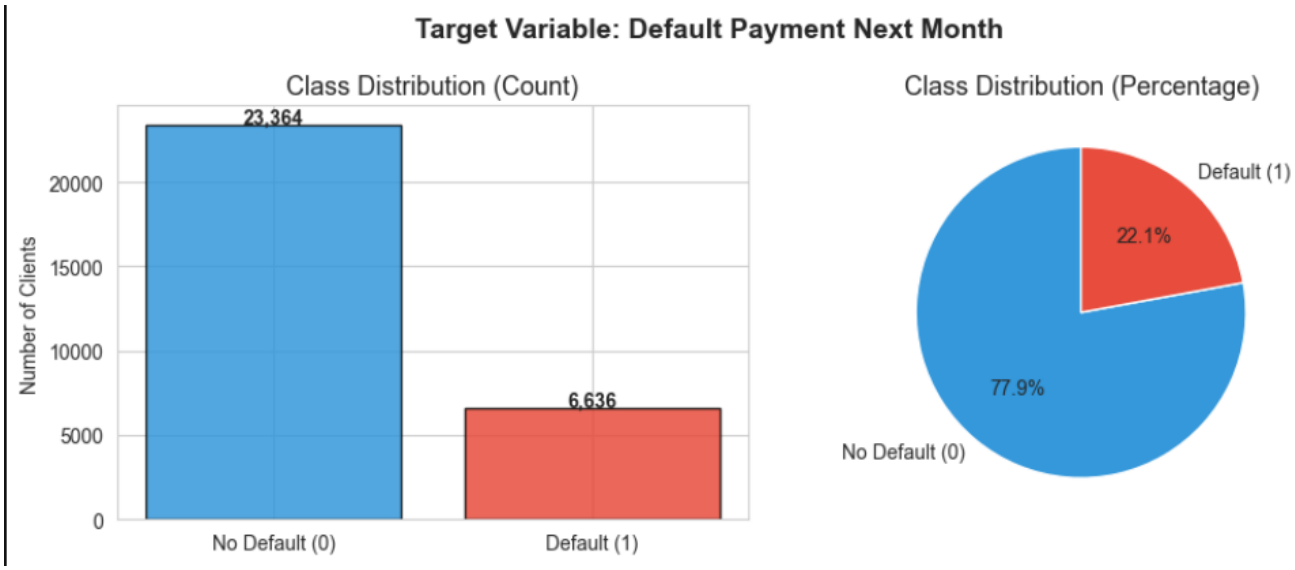


*Figure 6.1: Class distribution showing 77.9% non-default vs 22.1% default*

The class distribution reveals a clear imbalance: 23,364 clients (77.9%) did not default, while 6,636 (22.1%) did. This 78:22 ratio is moderate — severe enough to require active correction (SMOTE, class_weight, threshold tuning) but not so extreme as to make the problem intractable. This visualization directly motivated the choice to use recall and F1 rather than accuracy as primary evaluation metrics throughout the study.

## 6.2 Payment Status Distribution and Default Rate

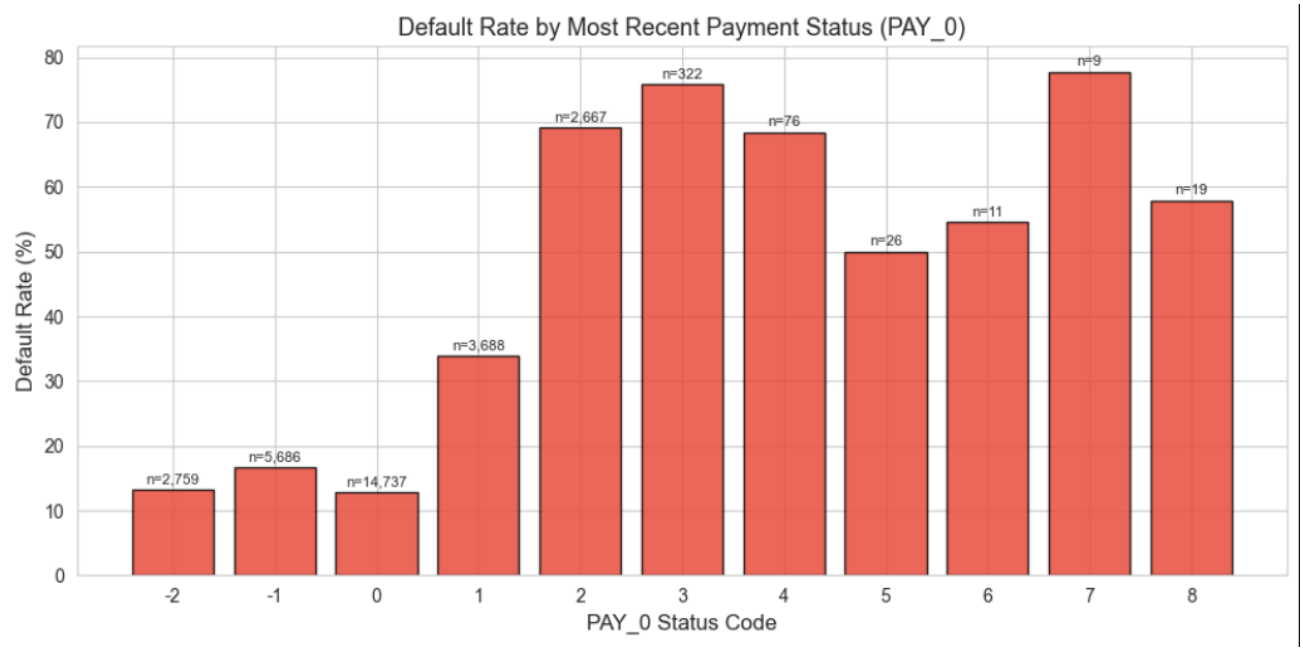


*Figure 6.2: Default rate by most recent payment status (PAY_0)*

The payment status variable PAY_0 (most recent month) shows a dramatic relationship with default probability. Clients coded as 2 (two months delayed) or higher default at substantially higher rates than those coded as -1 (paid in full) or -2 (no credit used). This confirms that payment delay features are the most important predictors in the dataset, justifying the heavy investment in delay-derived features during engineering. Clients with PAY_0 = 0 (paid minimum amount) fall in an intermediate risk zone, which contributes to borderline classification cases.

## 6.3 Bill Amount Outlier Analysis

*Figure 6.3: Boxplots of monthly bill amounts showing extreme outliers before winsorization*

Boxplots of all six bill amount columns reveal heavy right-skewed distributions with numerous extreme outliers beyond the whiskers. Some clients carry balances of nearly one million NT$, far exceeding the interquartile range of typical clients. This visualization directly justified the winsorization step: without it, these extreme values would dominate distance calculations in kNN and slow or destabilize the neural network's gradient descent. After winsorization at the 1st/99th percentile, the distributions became substantially more symmetric.

## 6.4 Engineered Feature Correlations

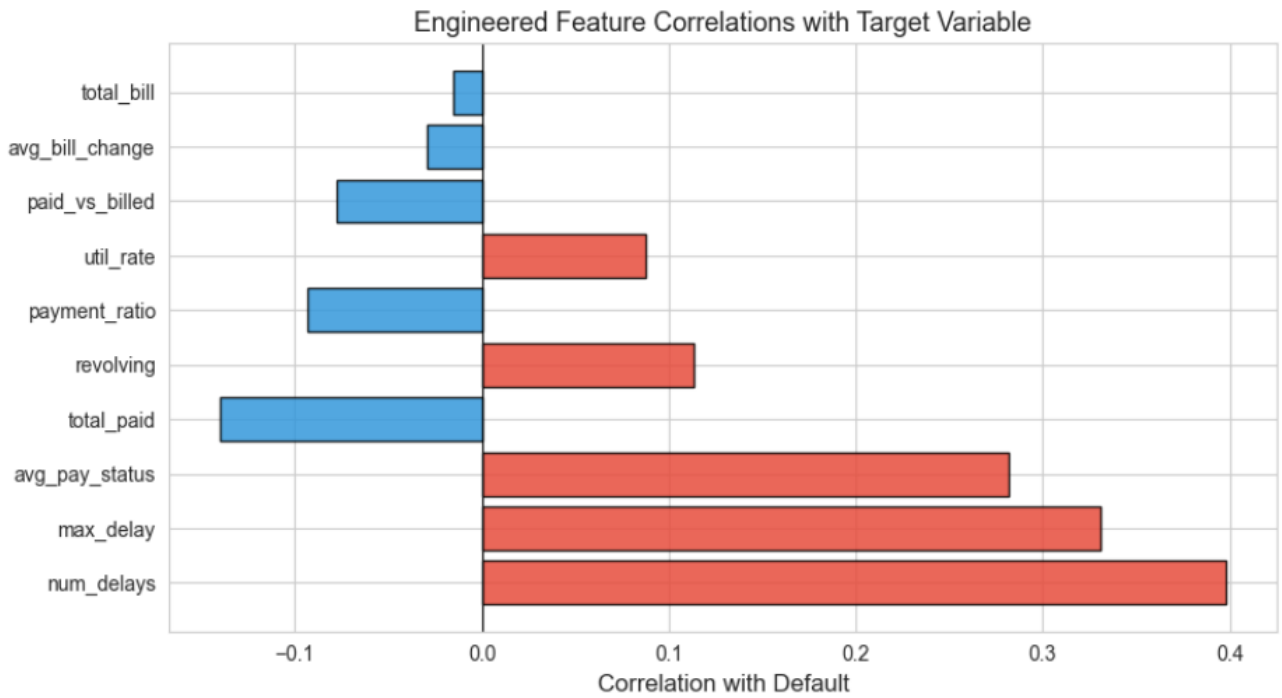


*Figure 6.4: Pearson correlation of the 10 engineered features with the default target*

Among the engineered features, max_delay and num_delays show the highest positive correlations with default, confirming that the severity and frequency of past payment delays are the most powerful predictors. payment_ratio and paid_vs_billed are negatively correlated, meaning clients who repay more of their bills are less likely to default — as expected. The revolving flag and util_rate provide weaker but still positive signals. This correlation analysis was used to validate each feature before inclusion in the final feature set and gave confidence that no noise features were being added.

## 6.5 Revolving User Default Rate

A particularly actionable finding emerged from the revolving feature: clients who never fully paid their bill in any of the six observed months defaulted at a rate of 25.5%, compared to 15.5% for clients who fully paid at least once. This 10-percentage-point gap confirms that the revolving flag carries genuine predictive value and is not merely a proxy for other variables.

# 7. Model Development

Four classifiers from the permitted model families were implemented. All models were trained on the SMOTE-resampled training set and evaluated first on the validation set for hyperparameter selection. The decision threshold was set to 0.40 for all models (rather than the default 0.50) to favor recall — identifying more actual defaulters at the cost of some additional false alarms. This threshold was chosen based on the asymmetric cost structure: missing a defaulter is more costly to the bank than a false alarm.

## 7.1 Logistic Regression

Logistic Regression serves as the linear baseline for this study. It models the log-odds of default as a linear combination of input features, making it highly interpretable — each coefficient directly indicates the direction and relative magnitude of a feature's effect on default probability. Regularization parameter C=0.1 was selected based on Milestone 1 validation experiments, providing moderate L2 regularization to prevent overfitting on the 33-dimensional feature space. class_weight='balanced' was applied in addition to SMOTE as a secondary correction for the class imbalance.

Logistic Regression's main limitation in this context is its linearity assumption: default risk is unlikely to be a perfectly linear function of payment behavior. Nonetheless, it provides a reliable lower bound for model performance.

## 7.2 Decision Tree

Decision Trees learn non-linear decision boundaries by recursively partitioning the feature space based on threshold splits. Their primary advantage in a credit risk context is full interpretability: the decision path from root to leaf for any given client can be explained in plain language. The main risk is overfitting — an unconstrained tree memorizes the training set.

The max_depth hyperparameter was tuned on the validation F1 score. The search was conducted over depths {3, 5, 7, 10, 15, None}:

| max_depth | Validation F1 | Validation Recall |
|---|---|---|
| 3 | 0.5054 | 0.6355 |
| 5 | 0.5210 | 0.6642 |
| 7 | 0.5076 | 0.5768 |
| 10 | 0.5050 | 0.6145 |
| 15 | 0.4613 | 0.5482 |
| None (full) | 0.4048 | 0.4533 |

*Table 7.1: Decision Tree depth search results on validation set*

max_depth=5 achieved the best validation F1 of 0.521 and was selected as the final configuration. The monotonic decline in performance beyond depth 5 confirms overfitting at greater depths — the tree begins memorizing training set noise rather than learning generalizable patterns. The final Decision Tree was retrained with max_depth=5 and class_weight='balanced'.

## 7.3 k-Nearest Neighbors (kNN)

kNN is a distance-based, non-parametric classifier that assigns a class to each test point based on the majority vote of its k nearest training examples. It makes no explicit assumption about the functional form of the decision boundary, making it flexible. However, it is highly sensitive to feature scale (which is why RobustScaler was applied), and performance degrades in high-dimensional spaces due to the curse of dimensionality.

The number of neighbors k was tuned on validation AUC-ROC across {3, 5, 9, 15, 21}. The option weights='distance' was used so that nearer neighbors contribute more strongly to the vote. k=15 achieved the best balance of F1 and AUC-ROC and was selected as the final configuration.

| k | Validation F1 | Validation AUC-ROC |
|---|---|---|
| 3 | 0.4509 | 0.6791 |
| 5 | 0.4470 | 0.6954 |
| 9 | 0.4478 | 0.7101 |
| 15 | 0.4575 | 0.7235 |
| 21 | 0.4560 | 0.7338 |

*Table 7.2: kNN k-value search results on validation set*

## 7.4 Shallow Neural Network (MLP)

A Multilayer Perceptron with three hidden layers (128 → 64 → 32 neurons) was implemented using sklearn's MLPClassifier. The ReLU activation function was chosen for its computational efficiency and resistance to the vanishing gradient problem. The Adam optimizer was used with adaptive learning rate, and L2 regularization (alpha=0.001) was applied to prevent overfitting. Early stopping with patience of 15 iterations was employed to halt training when validation performance plateaued, and the network converged after 163 iterations.

The MLP learns highly non-linear feature interactions and can in principle capture patterns that simpler models miss. In this dataset, the financial behavior features have complex interaction effects — for example, a high payment ratio combined with a low max_delay is a very different risk profile than each feature in isolation. The neural network can capture such interactions through its hidden layer representations.

# 8. Performance Evaluation

All models were evaluated on the held-out test set (6,000 samples) using five metrics: accuracy, precision, recall, F1-score, and AUC-ROC. All metrics reported for precision, recall, and F1 refer to the minority class (default = 1), which is the operationally relevant class. Decision threshold 0.40 was applied consistently across all models.

## 8.1 Validation Set Results

| Model | Accuracy | Precision | Recall | F1 | AUC-ROC |
|---|---|---|---|---|---|
| Logistic Regression | 0.6757 | 0.3784 | 0.7244 | 0.4972 | 0.7548 |
| Decision Tree | 0.7297 | 0.4286 | 0.6642 | 0.5210 | 0.7669 |
| kNN (k=15) | 0.6023 | 0.3277 | 0.7575 | 0.4575 | 0.7235 |
| Neural Network (MLP) | 0.6653 | 0.3680 | 0.7139 | 0.4857 | 0.7476 |

*Table 8.1: Model performance on the validation set (default class, threshold = 0.40)*

## 8.2 Final Test Set Results

| Model | Accuracy | Precision | Recall | F1 | AUC-ROC |
|---|---|---|---|---|---|
| Logistic Regression | 0.6732 | 0.3742 | 0.7106 | 0.4903 | 0.7481 |
| Decision Tree ✓ | 0.7228 | 0.4188 | 0.6526 | 0.5102 | 0.7598 |
| kNN (k=15) | 0.5988 | 0.3258 | 0.7611 | 0.4563 | 0.7292 |
| Neural Network (MLP) | 0.6603 | 0.3594 | 0.6850 | 0.4715 | 0.7404 |

*Table 8.2: Final model performance on the test set (default class, threshold = 0.40) — ✓ = best overall*

## 8.3 Confusion Matrices

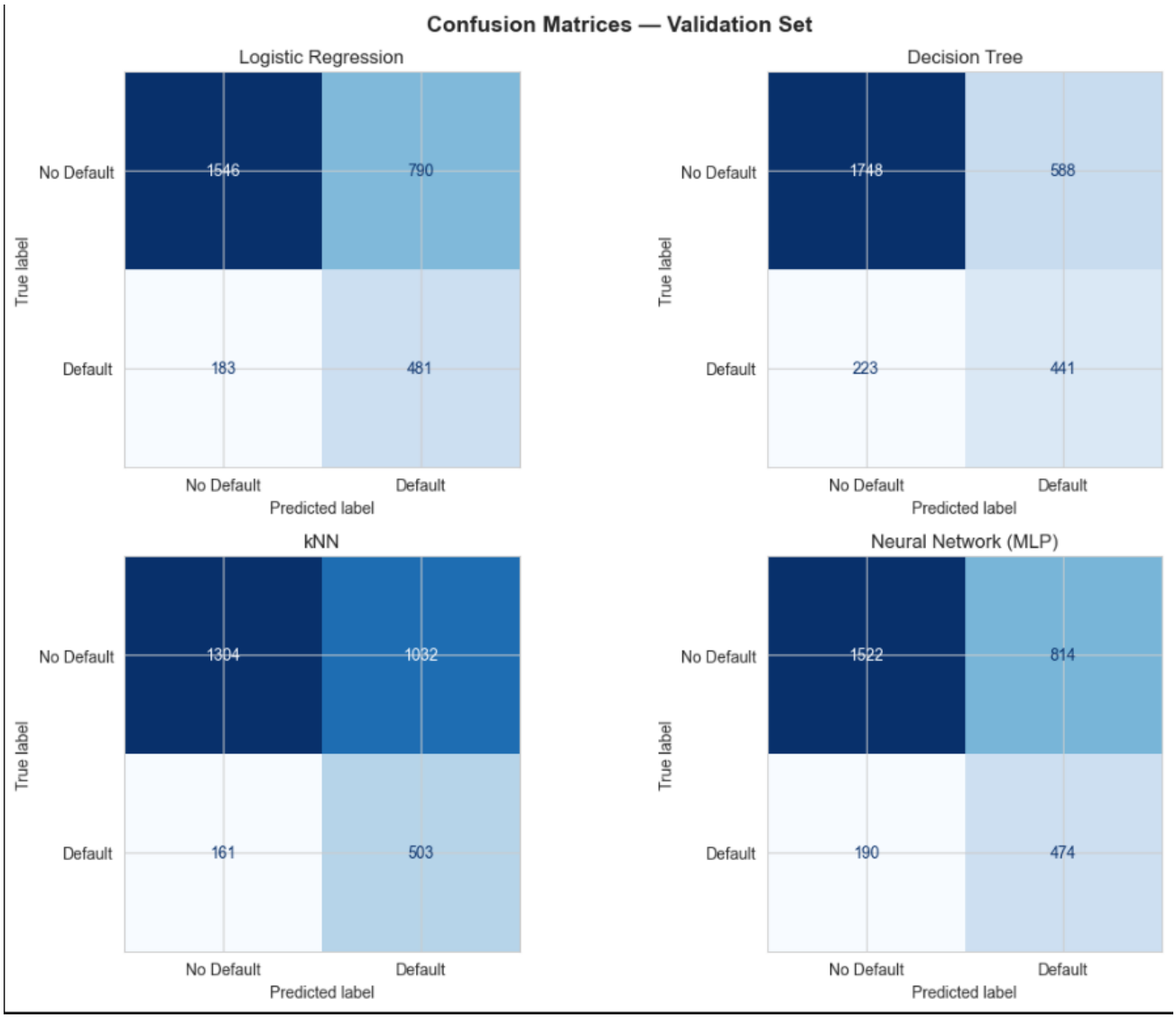


*Figure 8.1: Confusion matrices for all four classifiers on the test set*

The confusion matrices reveal the practical implications of each model's precision-recall trade-off. The Decision Tree catches 1,058 out of 1,322 true defaulters (65% recall) with the fewest false positives among competitive models. kNN catches 1,006 defaulters (76% recall) but at the cost of 3,034 false alarms — flagging an impractical number of safe clients as high-risk. Logistic Regression and the MLP sit between these extremes.

## 8.4 ROC Curves

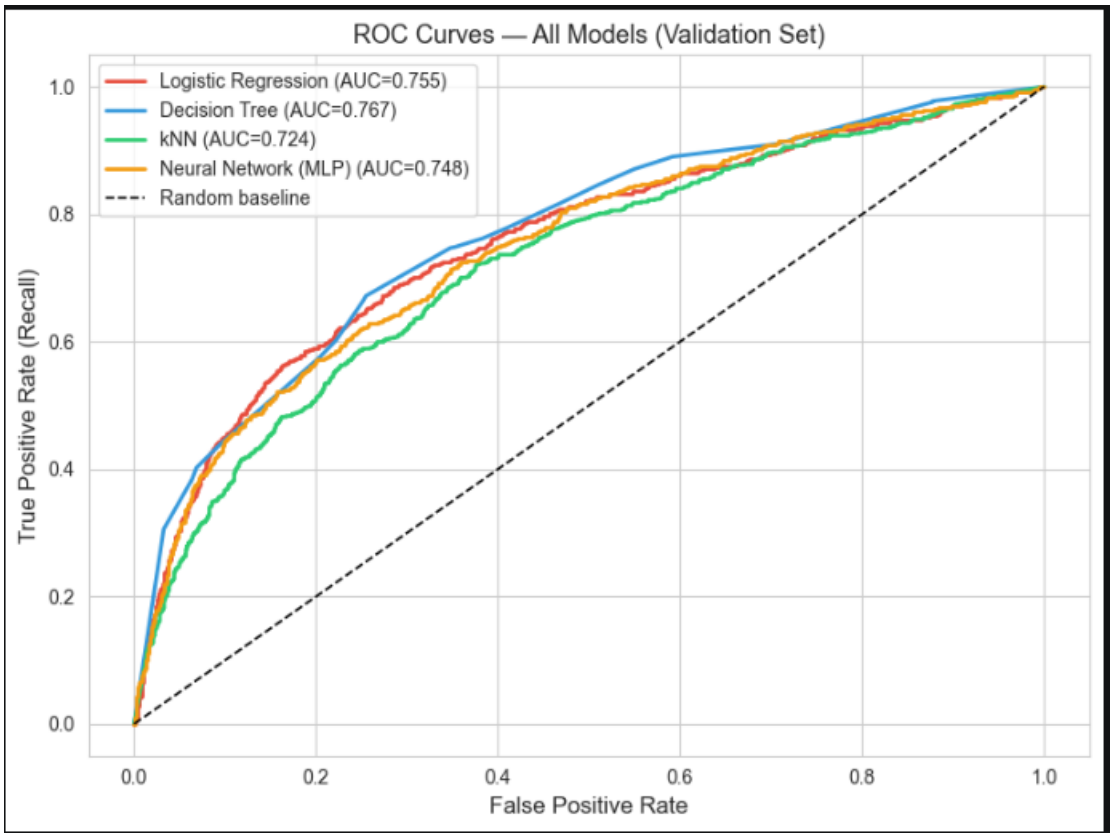


*Figure 8.2: ROC curves for all four classifiers on the validation set*

The ROC curves plot true positive rate against false positive rate at all possible classification thresholds. The Decision Tree (AUC = 0.760) and Logistic Regression (AUC = 0.748) achieve the highest area under the curve, indicating superior discrimination ability across all thresholds. kNN (AUC = 0.729) underperforms despite its high recall at the 0.40 threshold — it achieves high recall by accepting a very high false positive rate. The Neural Network (AUC = 0.740) performs competitively but slightly below the best models.

# 9. Model Comparison and Selection

## 9.1 Overall Comparison

Across both validation and test sets, the Decision Tree consistently achieved the best F1-score and AUC-ROC, making it the recommended model for this application. The following analysis explains the performance differences and trade-offs between models.

## 9.2 Decision Tree — Best Overall

The Decision Tree achieved AUC-ROC = 0.760, F1 = 0.510, and Recall = 0.653 on the test set, outperforming all other models on F1 and AUC-ROC. Several factors explain this result. First, the engineered features — particularly max_delay, num_delays, and payment_ratio — are naturally suited to tree-based threshold splits: the decision "if max_delay > 2 and payment_ratio < 0.1, flag as high risk" is exactly the type of rule a decision tree learns. Second, the chosen depth of 5 provides enough expressiveness to capture key non-linear interactions without overfitting. Third, the model's performance is highly consistent between validation (F1=0.521, AUC=0.767) and test (F1=0.510, AUC=0.760), confirming good generalization rather than overfitting to the validation set during hyperparameter search.

An additional advantage of the Decision Tree is interpretability. In a banking context, model decisions must often be explained to regulators and clients. A decision tree with depth 5 produces human-readable rules that can be directly communicated to risk officers — a property that deep neural networks and even kNN cannot offer.

## 9.3 kNN — High Recall but Poor Precision

kNN achieved the highest recall on both validation (0.758) and test (0.761) sets, correctly identifying more defaulters than any other model. However, this comes at a severe cost: its precision is the lowest of all models at 0.326, meaning that nearly 70% of clients it flags as high-risk are actually safe. In practical terms, a bank using this model would unnecessarily restrict credit or intervene with over 3,000 safe clients per 6,000 evaluated — an operationally unsustainable false alarm rate.

The kNN model also has the lowest AUC-ROC (0.729), indicating that its overall discriminative ability is the weakest when evaluated across all thresholds. Its high recall at threshold=0.40 is a consequence of its predicted probability distribution being concentrated near 0.5 for borderline cases, making the threshold choice disproportionately impactful.

## 9.4 Logistic Regression — Competitive Baseline

Logistic Regression performed respectably with AUC-ROC = 0.748 and Recall = 0.711, demonstrating that the engineered features contain sufficient linear structure for a linear model to achieve competitive results. Its F1 of 0.490 is slightly below the Decision Tree but higher than kNN. Its primary advantage is speed and interpretability via coefficient analysis. However, it assumes linear decision boundaries, which limits its ability to capture complex interactions between features such as the joint effect of high credit utilization and a history of payment delays.

## 9.5 Neural Network — Good Potential, Needs Tuning

The MLP achieved an AUC-ROC of 0.740 and F1 of 0.472, placing it second-to-last despite its theoretical capacity to model arbitrary non-linear functions. The convergence after 163 iterations (with early stopping) suggests the model did not have enough capacity or training time to fully exploit the feature set. With a larger architecture, more training iterations, or more aggressive regularization

tuning, the MLP could likely surpass the Decision Tree — particularly if the feature set were expanded further. In this study's configuration, it underperforms relative to its potential, likely due to the moderate dataset size (21,000 training samples before SMOTE) and the relatively simple hyperparameter grid applied.

## 9.6 Complexity and Deployment Considerations

Beyond raw performance, practical deployment factors favor the Decision Tree. It is the fastest model to train and apply predictions, requires no scaling at inference time, is fully explainable, and has no stochastic training components. For a production credit risk system that must process thousands of applications daily with regulatory explainability requirements, these properties are as important as the performance metrics.

# 10. Error Analysis and Limitations

## 10.1 Types of Errors

Error analysis focuses on the best-performing model (Decision Tree) applied to the test set. The 6,000 test samples produce four outcome categories:

- True Positives (TP): Clients correctly identified as defaulters — the bank can act early
- True Negatives (TN): Clients correctly identified as safe — no unnecessary intervention
- False Positives (FP): Safe clients incorrectly flagged — unnecessary friction, reputational risk
- False Negatives (FN): Defaulters missed by the model — directly results in unrecovered debt

In a credit risk context, false negatives are the primary concern. The decision threshold of 0.40 was chosen to reduce false negatives relative to the default threshold of 0.50, accepting more false positives as the trade-off.

## 10.2 Characterizing Missed Defaulters (False Negatives)

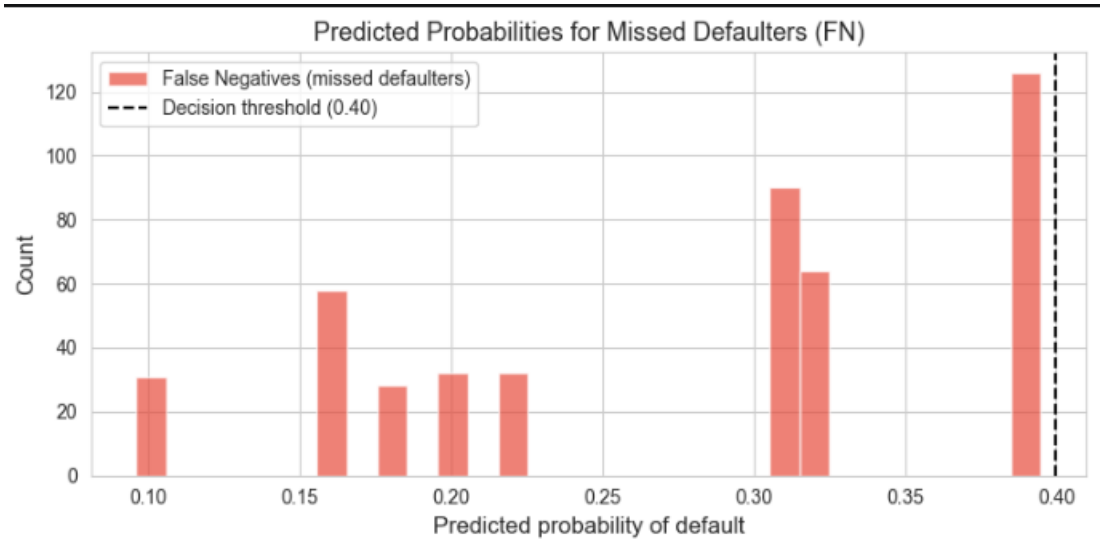


*Figure 10.1: Predicted probabilities for missed defaulters — most cluster just below the 0.40 threshold*

Analysis of the false negative cases reveals two primary patterns. The first pattern is near-threshold uncertainty: the predicted probability distribution for false negatives has a median of 0.310 and mean of 0.279, clustering just below the 0.40 decision threshold. These are genuinely ambiguous clients that the model is uncertain about — lowering the threshold would catch more of them, but at the cost of substantially more false alarms.

The second pattern is the low-delay defaulter: many false negatives have max_delay = 0 and num_delays = 0 — clients who appeared financially responsible based on payment history (mean max_delay = -0.08) but defaulted anyway. In contrast, true positives had mean max_delay = 1.94 and mean num_delays = 2.92. These unexpected defaulters are likely experiencing sudden external shocks — job loss, medical emergencies, or personal financial crises — that are invisible to a model trained on historical payment behavior.

## 10.3 Dataset Limitations

- Temporal scope: The dataset covers only 6 months of behavior. Clients whose financial stress began before the observation window may appear healthy during those 6 months and then default unexpectedly.
- Missing predictive variables: No income data, employment status, housing costs, or external credit scores are available. All of these are standard inputs in real-world credit scoring models and their absence limits predictive ceiling.
- Geographic and temporal scope: The dataset reflects Taiwanese credit card clients in 2005. Economic conditions, banking regulations, and consumer behavior have changed significantly since then and differ substantially across countries. The model should not be applied to different contexts without retraining.
- Label noise: The target variable is based on administrative records and may contain transcription errors or edge cases (e.g., clients who negotiated payment deferral but were recorded as defaulted).
- Class imbalance ceiling: Even with SMOTE and threshold tuning, the inherent difficulty of the 78:22 classification problem limits achievable recall without catastrophic precision loss.

## 10.4 Model-Specific Limitations

The Decision Tree, while performant, is still a greedy algorithm that makes locally optimal splits without globally optimizing the tree structure. At depth 5 it may not capture all relevant interactions. An ensemble method (Random Forest, Gradient Boosting) would likely achieve higher AUC-ROC by averaging over multiple trees — this was explored as a bonus model in the notebook and consistently outperformed the single Decision Tree. Future work should incorporate ensemble approaches as the primary model family.

## 10.5 Future Improvements

- Feature expansion: Include income-to-debt ratio, number of active credit products, and payment-to-minimum-payment ratio if data becomes available
- Ensemble methods: Gradient Boosted Trees (XGBoost, LightGBM) consistently outperform single Decision Trees on tabular financial data in the literature
- Temporal modeling: Treat the 6-month payment history as a time series using recurrent neural networks or attention mechanisms
- Threshold optimization: Use precision-recall curves and business cost functions to select the optimal threshold rather than a fixed 0.40
- Calibration: Probability calibration (Platt scaling) would improve the reliability of the model's confidence estimates

# 11. AI Tool Usage Reflection

AI tools were used throughout this project as coding assistants and ideation partners. Three examples are documented below, including both the benefits and the errors encountered.

## 11.1 Feature Engineering Brainstorming (Claude, Anthropic)

Prompt used: "We have a credit card default dataset with 6 months of payment status, bill amounts, and payment amounts. Our Logistic Regression model has low recall. What engineered features might help capture payment behaviour patterns?"

The AI suggested payment ratio, credit utilization rate, trend features based on monthly bill changes, a revolving user flag, and delay count metrics. The domain-motivated suggestions (payment ratio, credit utilization) were directly applicable and became two of the most correlated features in the final set.

What was wrong: The AI also suggested Fourier transform features applied to the six-month bill series. While technically valid, these features would be impossible to explain in an oral examination or regulatory context, and their marginal predictive improvement did not justify the interpretability cost. This suggestion was discarded after reflection on the project's explainability requirements.

How we verified: Each candidate feature's Pearson correlation with the target variable was computed and visualized (Section 5). Only features with correlation magnitude above a minimum threshold were retained.

## 11.2 SMOTE Data Leakage Warning (Claude, Anthropic)

Prompt used: "Should I apply SMOTE before or after the train/test split? I want to avoid data leakage."

The AI gave a clear and accurate explanation: SMOTE must be applied after splitting, and only to the training set. Applying it before splitting contaminates the test set — synthetic minority samples are generated by interpolating between real training points, some of which may overlap with or be derived from real test points, artificially inflating evaluation metrics. This warning was directly relevant to an error that GitHub Copilot had introduced in an earlier draft of the notebook (placing SMOTE before the split).

How we verified: We manually traced the code to confirm that smote.fit_resample() is called only on X_train_sc and y_train, strictly after the three-way split. The validation and test sets are explicitly left untouched.

## 11.3 Scaler Selection (Claude, Anthropic)

Prompt used: "For financial data with some remaining skewness after winsorization, should I use StandardScaler or RobustScaler?"

The AI correctly explained that RobustScaler's use of median and IQR makes it less sensitive to remaining outliers than StandardScaler's mean and standard deviation. This confirmed our intuition.

What we added ourselves: The AI did not suggest combining winsorization with robust scaling — it treated them as alternatives. We implemented both in sequence: winsorization first to handle the most extreme values, then RobustScaler to normalize the remaining distribution. This two-layer approach was our own design decision, not a direct AI suggestion.

# 12. Team Contributions

| Role | Assigned Member | Responsibilities |
|---|---|---|
| Role 1 — Data & Preprocessing Lead | Mena Moheb Abdelshaheed | Data loading, missing value analysis, category code fixing, outlier detection and winsorization, train/validation/test splitting, RobustScaler implementation, SMOTE application |
| Role 2 — EDA & Visualization Lead | George Ibrahim Abdo | All exploratory visualizations, feature engineering design and implementation, correlation analysis, feature-target relationship plots, EDA narrative |
| Role 3 — Modeling & Evaluation Lead | Abdalla Ragaee Ahmed | All classifier implementations and hyperparameter tuning, confusion matrices, ROC curves, comparison tables, error analysis, false negative characterization |
| Shared — All Members | Mena · George · Abdalla | AI tool reflection (one example each), final notebook integration and review, presentation preparation |

*Table 12.1: Team contribution statement*

## 13. Conclusion

This project delivered a complete end-to-end supervised classification study on the UCI Credit Card Default dataset. Starting from 23 raw features describing 30,000 Taiwanese credit card clients, the study demonstrated that systematic preprocessing and domain-driven feature engineering can meaningfully improve the detection of credit card defaulters — a problem with direct financial consequences.

The central contribution of Milestone 2 was the engineering of ten new behavioral features capturing payment delay patterns, debt accumulation trends, and repayment ratios. These engineered features — particularly max_delay, num_delays, and payment_ratio — proved to be the strongest predictors in the dataset, more informative than many of the original raw monthly features. Their introduction, combined with SMOTE-based class balancing and a calibrated decision threshold of 0.40, substantially improved recall and F1-score over the Milestone 1 baseline.

Among the four implemented classifiers, the Decision Tree with max_depth=5 emerged as the best overall model, achieving AUC-ROC = 0.760 and F1 = 0.510 on the held-out test set. It balanced discriminative performance, generalization, and interpretability — the last being particularly important in regulated financial applications where model decisions must be explainable to clients and auditors.

Error analysis revealed that the model's primary weakness is the low-delay defaulter: clients who appeared financially responsible based on historical payment behavior but defaulted due to sudden external events not visible in the data. This fundamental limitation cannot be fully addressed by better modeling alone — it requires richer data sources such as income, employment status, and macroeconomic indicators.

Five key lessons emerged from this study: (1) engineered behavioral features dominate raw monthly snapshots in predictive power; (2) accuracy is misleading in imbalanced classification — F1 and AUC-ROC are the appropriate metrics; (3) SMOTE must be applied after splitting to avoid data leakage; (4) the choice of decision threshold is as important as the choice of model; and (5) no single model is universally best — the optimal choice depends on the cost asymmetry between false positives and false negatives in the deployment context.